

Indian Institute of Technology, Madras
Parallel Scientific Computing - ID5130

Kanak Agarwal

March 16, 2026

1 Riemann Integration

The integral is given by,

$$I = \int_0^4 \int_0^3 \int_0^2 (4x^3 + xy^2 + 5y + yz + 6z) \, dz dy dx \quad (1)$$

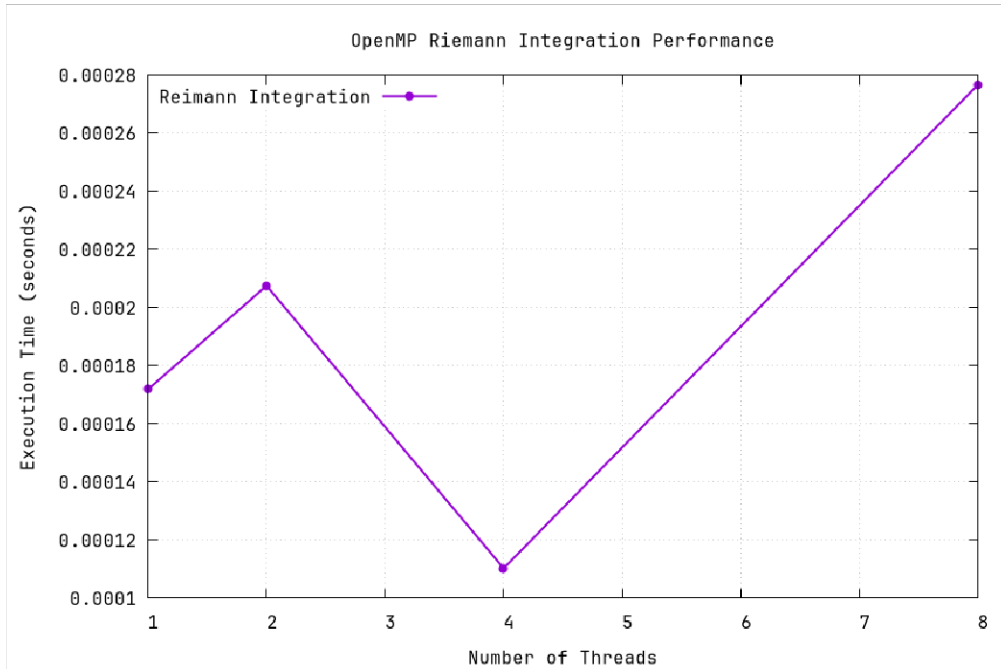
The exact value of the integral is 2040. In Riemann integration, the value of an integral is approximated by Riemann sums, given by,

$$I \approx \sum_{i=1}^{N_x} \sum_{j=1}^{N_y} \sum_{k=1}^{N_z} f(x_i, y_j, z_k) \Delta x \Delta y \Delta z \quad (2)$$

This is implemented in FORTRAN and the results are as follows for all threads with $N_x = 48, N_y = 36, N_z = 24$,

Numerical Integral = 2039.6388888888823
Analytical Integral = 2040.0000000000000
Relative Error = 1.7701525054788222E-002 %

The parallel performance of the implementation with OpenMP is depicted in the figure below.



It is clear that after 4 threads, the performance degrades. This is due to the additional overhead of a parallel implementation, and the optimum configuration is 4 threads which has a speedup of $1.5\times$ compared to the serial implementation.

2 Monte Carlo Integration

In the Monte Carlo method, the integral is approximated by random sampling of points in the domain of integration. The value of the integral is given by,

$$I \approx V\bar{f} \pm Vf_{rms} \quad (3)$$

where,

$$\bar{f} = \frac{1}{N} \sum_{i=1}^N f(x_i, y_i, z_i) \quad (4)$$

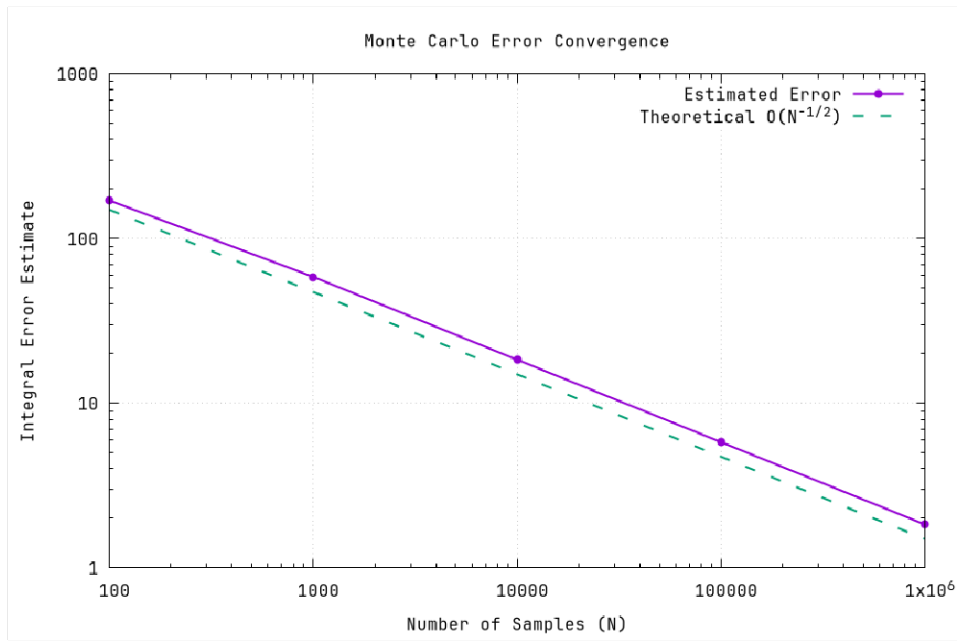
and,

$$f_{rms} = \sqrt{\bar{f^2} - \bar{f}^2} \quad (5)$$

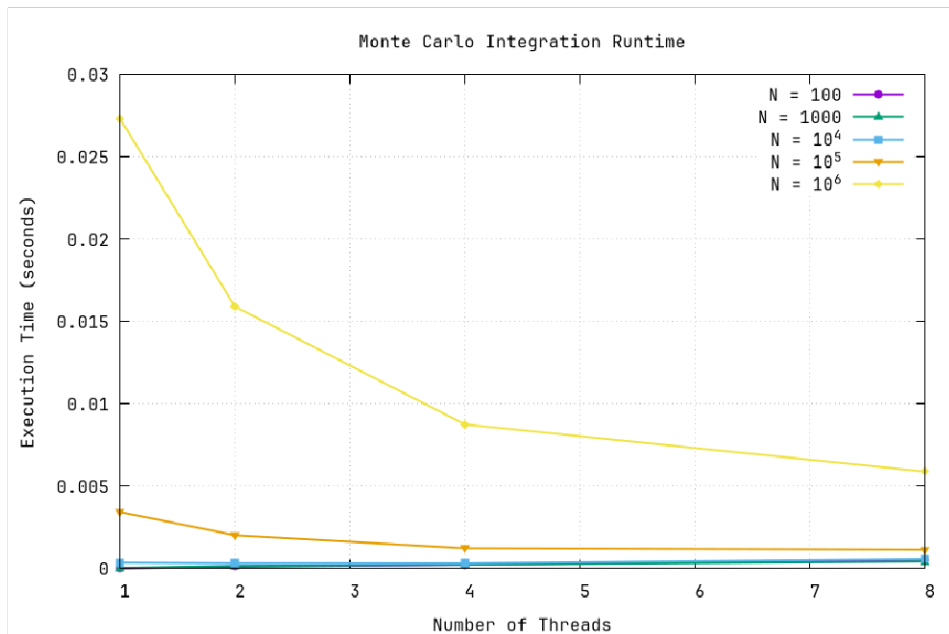
where,

$$\bar{f^2} = \frac{1}{N} \sum_{i=1}^N f(x_i, y_i, z_i)^2 \quad (6)$$

The results of this method with N is given by the following figure,



Its parallel performance with OpenMP is depicted in the figure below.



3 Jacobi Method for Poisson's Equation

Poisson's equation along with its boundary conditions are given by,

$$\nabla^2 \phi = -q, \quad \phi(\pm 1, y) = \phi(x, \pm 1) = 0 \quad (7)$$

The solution to this equation is obtained by the Jacobi method, which is an iterative method for solving linear systems. The equation is discretized as,

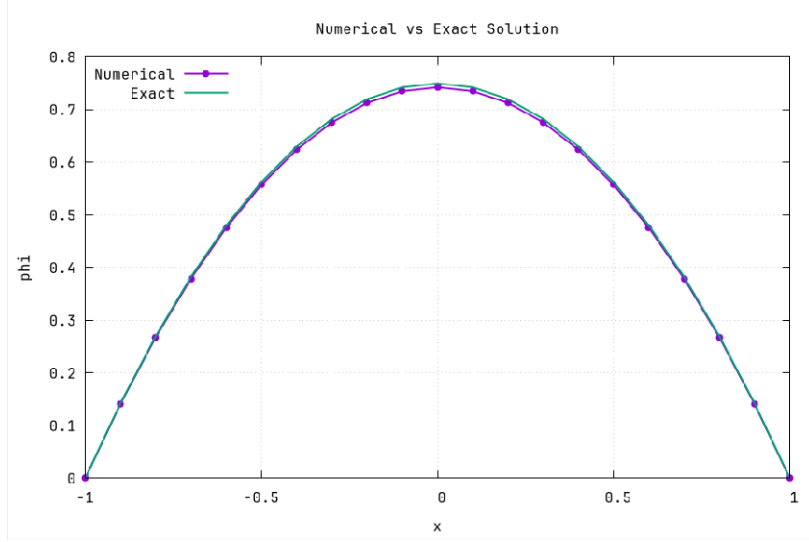
$$\phi_{i,j} = \frac{1}{4}(\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1} + q_{i,j}\Delta^2) \quad (8)$$

The analytical solution to this equation is given by,

$$\phi = (x^2 - 1)(y^2 - 1) \quad (9)$$

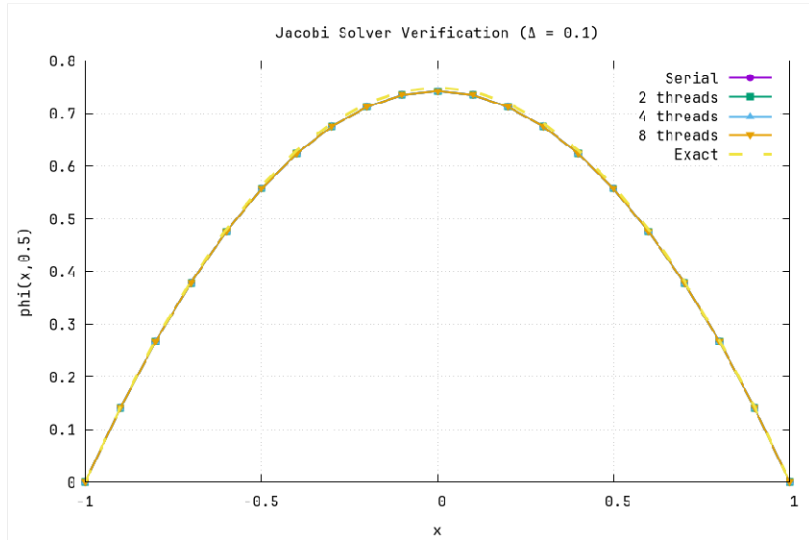
3.1 Serial Implementation

The results of the serial implementation of the Jacobi method is given by the following figure (350 iterations to converge within 1% relative error),

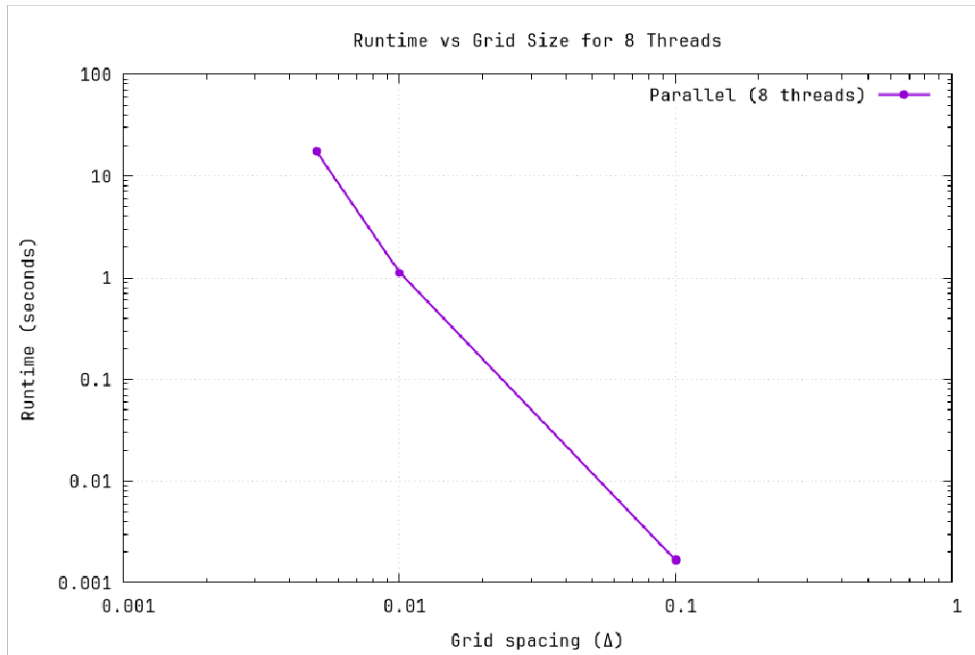


3.2 Parallel Implementation

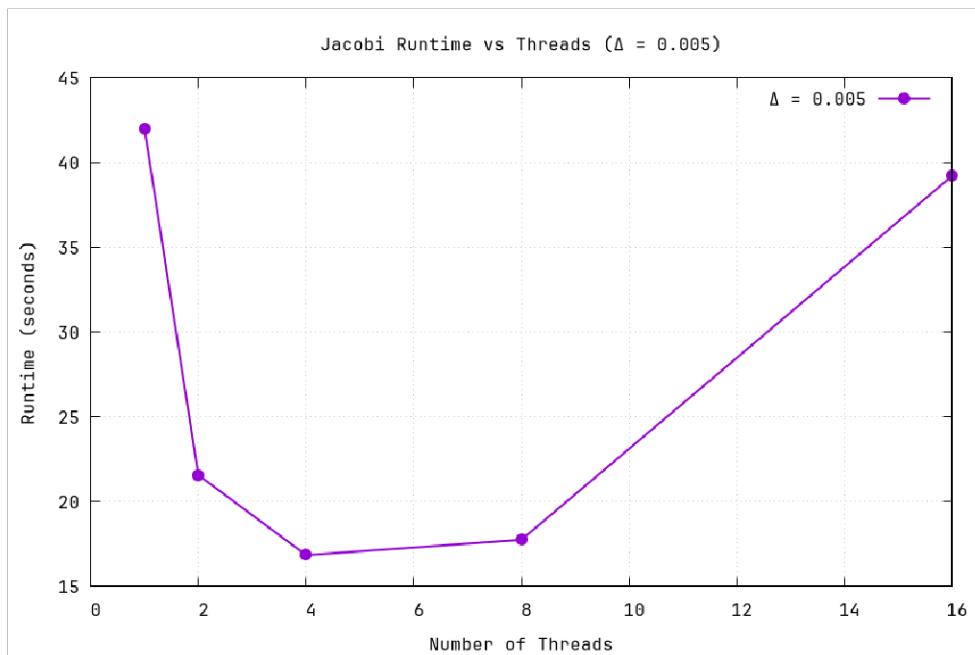
The parallel implementation of the Jacobi method is done using **OpenMP**. The solution is same as the serial implementation, which is verified by the following figure,



We now run different delta values with 8 threads to compare the runtime of the parallel implementation. The results are given by the following figure,



As expected, the runtime increases with decreasing delta value, which is due to the increase in the number of grid points and hence the number of iterations required to converge. A similar plot is obtained for a fixed delta value with different number of threads, which is given by the following figure,



The optimum number of threads is 4, for which a speedup of $\approx 2.5\times$ is obtained compared to the serial implementation. The performance degrades after 4 threads due to the additional overhead of a parallel implementation.